Claude Code × Notion・GitHub・Figma・Docker

MCP連携とサブエージェント

MCP integration and subagents for Claude Code (Windows)

公式ドキュメント準拠

講師用（ノート付き）

うさうさ研修工房　YUKIKO

この講座のゴールと対象

学べること

- ✔ MCPが何かを説明できる
- ✔ Notion・GitHub・Figma・Dockerを接続できる
- ✔ サブエージェントに役割を分けられる
- ✔ 安全に使う考え方を持てる

対象と前提

- ・Windows＋PowerShellが使える
- ・Claude Codeが動く
- ・各サービスのアカウントがある
- ・Docker Desktopが入っている

学ぶメリット：何が変わる？

学ぶ前	学んだ後
・仕様・コード・デザインを、コピーして貼る ・1つの会話が、情報で重くなる ・どのツールを許可したか、あいまい	・Claudeが直接読める（MCP） ・役割ごとのサブに任せて、会話を軽く ・権限を絞って、安全に任せる

直接つながる
公式の機能

会話が軽い
公式の考え方

権限を絞れる
公式の機能

役割を分けられる
うさうさの提案

考えてみよう

Q1 | 問いかけ

Q1

仕様はNotion、コードはGitHub、デザインはFigma、環境はDocker。どれをClaudeに読ませ、どれを書かせますか？

A 全部、読み取りだけ

B 読んで、書くのも許可

C 書き込みだけ許可

▶ **動画を一時停止して、30秒考えてみましょう。 最初は、どれが安全でしょう？**

MCPとは：外部のツールにつなぐ

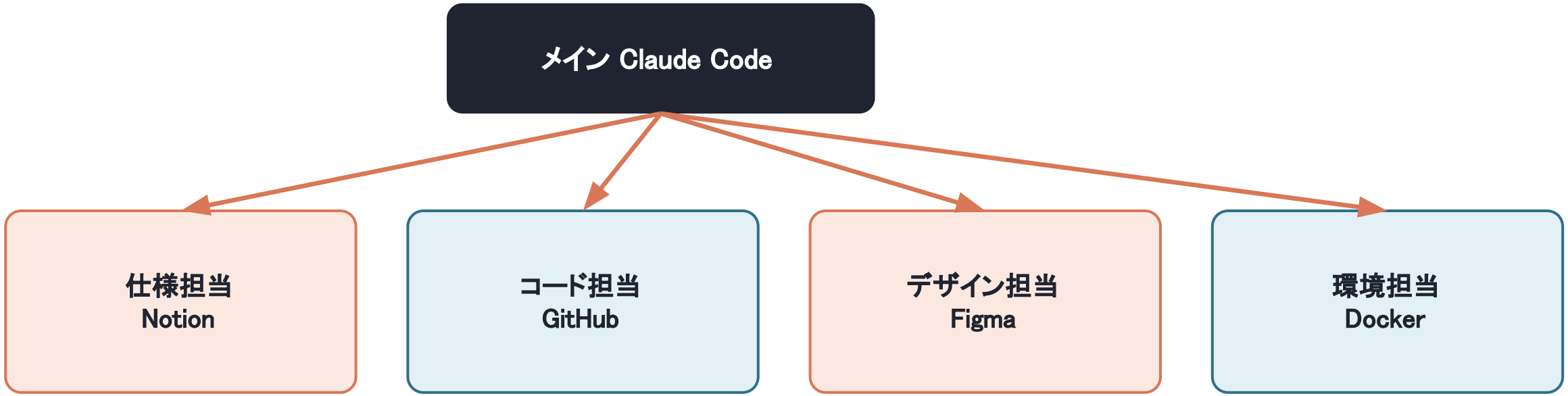



```
claude mcp add ...      #  追加
claude mcp list         #  一覧・接続確認
/mcp                    # Claude Code 内で状態・認証
```

MCP gives Claude Code access to external tools, data, and APIs.

MCPは、Claude Codeに外部のツール・データ・APIを使わせる仕組みです。

全体像：メインが指揮、サブが担当



サブエージェントは、独立した作業場で働き、結果の要約だけを返します。
役割分担の図は、うさうさ研修工房の提案です。

考えてみよう

Q2 | 問いかけ

Q2

4つのサービスを、全部1つの会話に入れると、何が起きそうですか？

- **A** 会話が重くなる
- **B** 特に変わらない
- **C** 速くなる

▶ **動画を一時停止して、30秒考えてみましょう。 大量の結果は、どこに積み上がるでしょう？**

準備：始める前のチェック

✔ **Claude Code**
ターミナルで claude が動く

✔ **アカウント**
Notion・GitHub・Figma

✔ **Docker Desktop**
MCP Toolkitは公式で4.62以降の画面が前提

✔ **PowerShell**
Windowsのターミナル

Windows PowerShell: claude --version で確認

Notionを接続する


```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

1 追加

・上のコマンドを実行
・公式（Claude Code）の例

2 認証

・Claude Code内で /mcp
・ブラウザの認証に従う

3 確認

・/mcp で接続済みか
・読み取りの依頼を1つ

提案：最初は読み取りだけ。Notion側の公式ドキュメントで権限を確認。

GitHubを接続する：2つの方法

A リモート（PAT）

```
claude mcp add --transport http github
https://api.githubcopilot.com/mcp -H "Authorization: Bearer
YOUR_GITHUB_PAT"
```

・GitHubが推奨するのはリモート
・追加後はClaude Codeを再起動
・claude mcp list で確認

B ローカル（Docker）

```
claude mcp add github -e
GITHUB_PERSONAL_ACCESS_TOKEN=YOUR_GITHUB_PAT -- docker run
-i --rm -e GITHUB_PERSONAL_ACCESS_TOKEN
ghcr.io/github/github-mcp-server
```

注意
・ローカルはDocker Desktopの起動が必要
・トークンは必要な権限だけ。貼らない

Figmaを接続する

A プラグイン（推奨）

```
claude plugin install figma@claude-plugins-official
```

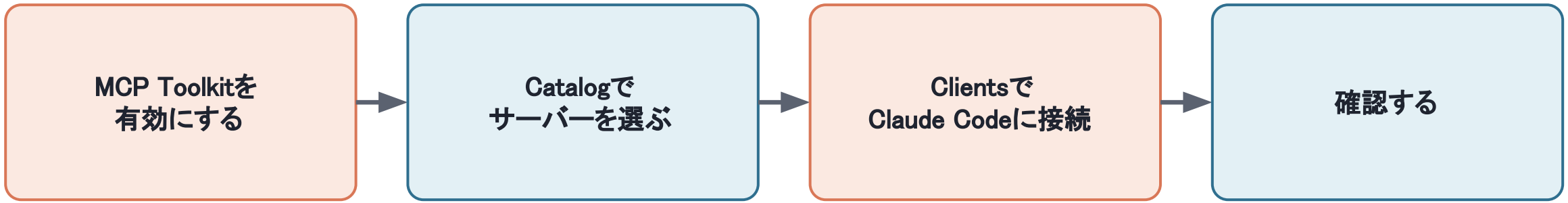
B 手動

```
claude mcp add --transport http figma
https://mcp.figma.com/mcp
```

使い方のポイント

・リモートサーバーが推奨（デスクトップ版は特定の組織向け）
・手動は、全プロジェクトで使うなら --scope user
・再起動後、/mcp で figma を選んで認証し、接続を確認
・リンク渡し：Figmaのフレームへのリンクを、依頼文に入れる

Docker Desktop：MCP Toolkit




```
claude mcp list
MCP_DOCKER: docker mcp gateway run - ✓ Connected
```

手動で接続する場合
クライアント側で docker mcp gateway run --profile（プロファイル名） を実行する設定にします。 提案：GitHubは「直接」か「Docker経由」のどちらか1つに。

考えてみよう

Q3 | 問いかけ

Q3

サブエージェントは、途中で質問できません。依頼文に、書き忘れると一番困ることは何でしょう？

A 範囲外（触らないもの）

B 完了条件

C 出力形式

▶ **動画を一時停止して、30秒考えてみましょう。 全部書くのが原則です。**

サブエージェントの基本と例

公式の4点

- 独立した作業場（コンテキスト）で動く
- 結果の要約だけを返す
- 途中で質問できない
- Markdownで作る（.claude/agents/）
- tools と mcpServers で絞れる

```
---
name: figma-reader
description: Figmaのリンクから
  実装に必要な要点をまとめる。読み取り専用。
mcpServers:
  - figma
---
目的：リンクの画面の構成・色・文字を整理
範囲外：Figmaへの書き込み・コード変更
出力形式：箇条書き
完了条件：依頼された画面をすべて挙げる
判断できない点は「要確認」
```

役割分担の表（提案）

サブ	使うMCP	任せること	最初の権限
仕様担当	Notion	仕様ページの要約	読み取りのみ
コード担当	GitHub	Issue・PRの確認、変更案	読み取り中心
デザイン担当	Figma	デザインの読み取り	読み取りのみ
環境担当	Docker MCP Toolkit	環境の確認	確認のみ

自分の仕事に合わせて、1行書き換えてみましょう。

並列で呼ぶ：1つの表にまとめる


```
仕様担当でNotionの「ログイン画面仕様」を要約し、
デザイン担当で次のFigmaリンクの画面を読み取り、
コード担当で関連するIssueを確認して。
最後に3つの結果を1つの表にまとめて。（互いに独立なので並列で）
```

期待する結果

✔ サブが3つ動く
✔ 各サブが要約だけ返す
✔ メインが1つの表にまとめる

注意

・並列は、互いに独立した調査だけ
・各サブが使用量を消費する

テストと確認

- ☐ claude mcp list で、4つの接続が出る
- ☐ /mcp で、それぞれ接続済みと表示される
- ☐ 各サービスに、読み取りだけの依頼を1つずつ
- ☐ サブを呼び、要約だけが返る
- ☐ 並列の依頼で、3つの結果が1つにまとまる

うまくいかないとき

接続が Failed・認証できない	/mcp から再認証。GitHubはPAT・URLを確認
GitHubのリモートでOAuthが通らない	PATをヘッダーで渡す方法（方法A）を使う
ツールが多くて会話が重い	サブ専用にする。未使用のMCPを無効に
追加したのに反映されない	Claude Codeを再起動。新しい会話で /mcp
Dockerで見えない	Docker Desktop起動・Toolkit有効・Clientsで接続

安全の考え方：5つ

最小の権限	読み取りから始める
トークンを書かない	共有の設定ファイル（.mcp.json）にも入れない
信頼できるサーバーだけ	第三者のサーバーは自己責任
外部への操作は人が確認	Issue作成・ページ更新など
入口を1つに	GitHubを二重に入れない

考えてみよう

Q4 | 問いかけ

Q4

読み取りに慣れました。書き込みも許可したくなったとき、どんな条件がそろったら許可しますか？

- **A** 確認の仕組みがある
- **B** 試す環境がある
- **C** 人が最終確認する

▶ **動画を一時停止して、30秒考えてみましょう。 正解は1つではありません。**

自己チェック：説明できますか？

振り返り

- ☐ MCPが何かを説明できる
- ☐ Notionを接続できた
- ☐ Figmaをプラグインか手動でつなげた
- ☐ サブエージェントの基本4点を言える
- ☐ claude mcp add / list / /mcp を使える
- ☐ GitHubの2方法を説明できる
- ☐ Docker MCP Toolkitで接続を確認できた
- ☐ 安全の5つの考え方を言える

Q　最初の問い「どれを読ませ、どれを書かせるか」に、今ならどう答えますか？

まとめ / Summary

MCPは追加して、listと /mcp で確認
Add with claude mcp add; verify with list and /mcp

サブエージェントで役割を分ける
Split roles across subagents

読み取りから始め、権限を広げる
Start read-only; widen permissions later

参考（公式）: code.claude.com/docs/en/mcp ／ sub-agents ／ github.com/github/github-mcp-server
developers.figma.com/docs/figma-mcp-server ／ docs.docker.com/ai/mcp-catalog-and-toolkit
役割分担・安全の考え方は、うさうさ研修工房の提案です。コマンドは公式で確認し、実機で試してください。

講師用：進行表（約105分）

講師用

枚	内容	分	楽/苦
1	導入	2	楽
2	ゴールと対象	3	楽
3	学ぶメリット	4	楽
4	Q1 読ませる・書かせる	4	楽
5	MCPとは	5	苦
6	全体像	6	苦
7	Q2 全部1会話に入れると	4	楽
8	準備	5	苦
9	Notionを接続	6	苦
10	GitHubを接続	7	苦
11	Figmaを接続	6	苦

枚	内容	分	楽/苦
12	Docker MCP Toolkit	7	楽
13	Q3 依頼文に何を書く	4	楽
14	サブエージェントの基本	6	苦
15	役割分担の表	5	苦
16	並列で呼ぶ	5	楽
17	テストと確認	6	楽
18	うまくいかないとき	5	苦
19	安全の考え方	4	苦
20	Q4 書き込みを許可する条件	4	楽
21	自己チェック	4	楽
22	まとめ	3	楽

時間配分は仮の目安（休憩を含め、2時間枠で調整）。押したときは⑱⑲の説明を短くします。